

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

struct Token {
    char value[50];
};

int main() {
    char input[200];
    struct Token tokens[50];
    int count = 0;
    int i = 0, j = 0;

    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    while (input[i] != '\0') {

        while (isspace(input[i])) {
            i++;
        }

        if (input[i] == '\0')
            break;

        j = 0;

        while (!isspace(input[i]) && input[i] != '\0') {
            tokens[count].value[j++] = input[i++];
        }

        tokens[count].value[j] = '\0';
        count++;
    }

    printf("\nTokens:\n");
```

```

    for (i = 0; i < count; i++) {
        printf("Token %d: %s\n", i + 1, tokens[i].value);
    }

    return 0;
}

```

```

sravya@Sravya1955:~/os$ nano tokenizer.c
sravya@Sravya1955:~/os$ gcc tokenizer.c -o tokenizer
sravya@Sravya1955:~/os$ ./tokenizer
Enter a command: copy file1.txt file2.txt

Tokens:
Token 1: copy
Token 2: file1.txt
Token 3: file2.txt
sravya@Sravya1955:~/os$ |

```

```

Token 3: file2.txt
sravya@Sravya1955:~/os$ ls
1      4.c      exec      os.c      os1.1.6.c  os1.c     os5.c     program7  shell.c      signal2.c  signal6  std.c
1.1.1c FileDescriptors  fifo_project  os1.1.1.c  os1.1.7.c  os1.xc    os6.c     program7.c  shell_history  signal3  signal6.c  std.c.save
1.c    LinuxLab      file1.txt    os1.1.2.c  os1.1.8.c  os14.c    os7.c     project     shell_history.c  signal3.c  signal7  test.txt
2      MyProject     filecopy     os1.1.3.c  os1.1.c    os15.c    os8.c     quotes     shell_history.c.save  signal4  signal7.c  tokenizer
2.c    bin          filecopy.c   os1.1.4.c.save  os1.2.c    os2.c     parser    quotes.c    signal1     signal4.c  source.txt
3      destination.txt  new          os1.1.5.c    os1.3.c    os3.c     parser.c   sample.txt  signal1.c   signal5  src      tokenizer.c
3.c    docs        op.save      os1.1.6     os1.4.c    os4.c     program1.c  shell       signal2     signal5.c  std
sravya@Sravya1955:~/os$

```

```
sravya@Sravya1955:~/os$ nano tokenizer.c
sravya@Sravya1955:~/os$ gcc tokenizer.c -o tokenizer
sravya@Sravya1955:~/os$ ./tokenizer
Enter a command: copy file1.txt file2.txt

Tokens:
Token 1: copy
Token 2: file1.txt
Token 3: file2.txt
Token stream is valid.
sravya@Sravya1955:~/os$ ./tokenizer
Enter a command: delete file1.txt file2.txt

Tokens:
Token 1: delete
Token 2: file1.txt
Token 3: file2.txt
Error: Command must start with 'copy'
sravya@Sravya1955:~/os$ |
```

```
GNU nano 7.2
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 100
#define MAX_TOKENS 20

struct ParseTree {
    char command[50];
    char source[50];
    char destination[50];
};

int parse(char *tokens[], int count, struct ParseTree *tree) {
    if (count == 0) {
        printf("Error: Empty command!\n");
        return 0;
    }

    if (strcmp(tokens[0], "copy") != 0) {
        printf("Syntax Error: Command must be 'copy'\n");
        return 0;
    }

    if (count != 3) {
        printf("Syntax Error: copy requires two arguments\n");
        return 0;
    }

    strcpy(tree->command, tokens[0]);
    strcpy(tree->source, tokens[1]);
    strcpy(tree->destination, tokens[2]);

    return 1;
}

int main() {
```

```
int main() {

    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int count;

    while (1) {

        printf("myshell> ");

        if (fgets(input, MAX_INPUT, stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0) {
            printf("Error: Empty command!\n");
            continue;
        }

        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        count = 0;

        char *token = strtok(input, " \t");

        while (token != NULL && count < MAX_TOKENS) {
            tokens[count++] = token;
            token = strtok(NULL, " \t");
        }

        printf("\nTokens:\n");
    }
}
```

```

char *token = strtok(input, " \t");

while (token != NULL && count < MAX_TOKENS) {
    tokens[count++] = token;
    token = strtok(NULL, " \t");
}

printf("\nTokens:\n");

for (int i = 0; i < count; i++) {
    printf("%d : %s\n", i + 1, tokens[i]);
}

struct ParseTree tree;

if (parse(tokens, count, &tree)) {

    printf("\nParsing Successful!\n");

    printf("\nParse Tree:\n");
    printf("      %s\n", tree.command);
    printf("      /  \\\n");
    printf("    %s    %s\n",
        tree.source, tree.destination);

    printf("\nExecution Structure:\n");
    printf("Command      : %s\n", tree.command);
    printf("Source       : %s\n", tree.source);
    printf("Destination  : %s\n\n",
        tree.destination);
}

return 0;
}

```

```
sravya@Sravya1955:~/os$ nano parser1.c
sravya@Sravya1955:~/os$ gcc parser1.c -o parser1
sravya@Sravya1955:~/os$ ./parser1
```

Parse Successful!

Parse Tree:

```
      copy
     /  \
file1.txt file2.txt
```

Execution Structure:

```
Command   : copy
Source    : file1.txt
Destination: file2.txt
```